



FaceAttend

EDGE AI ATTENDANCE SYSTEM · DATALAKE 3.0 SUBMISSION

A Python desktop proof-of-concept for real-time, offline face recognition with blink-based liveness detection — zero cloud dependency, 40.70 MB model footprint, LFW 99.31% accuracy.



Solving Three Hard Attendance Problems

Proxy Fraud

Static photograph attacks blocked by EAR blink liveness with anti-tilt spoof guard — a photo cannot simulate a real blink.

Unreliable Connectivity

All processing is local. No network calls during recognition. Designed for remote sites and factory floors where cloud systems fail silently.

Lighting Variance

MediaPipe BlazeFace and FaceLandmarker handle harsh sunlight, deep shadows, and low-light outdoor deployments robustly.

Directory Structure & Model Footprint

Artifact Inventory

File	Size
blaze_face_short_range.tflite	0.22 MB
face_landmarker.task	3.58 MB
face_recognition_sface_2021dec.onnx	36.90 MB
Total	40.70 MB

Detection + liveness stack: **3.80 MB**. SFace embedding model: **36.90 MB**.

Single-File Architecture

All logic resides in `face_recognition_app.py` (775 lines). Models auto-download at startup via `ensure_models()` — the app calls `sys.exit(1)` if any file is missing, ensuring it never runs with incomplete artifacts.

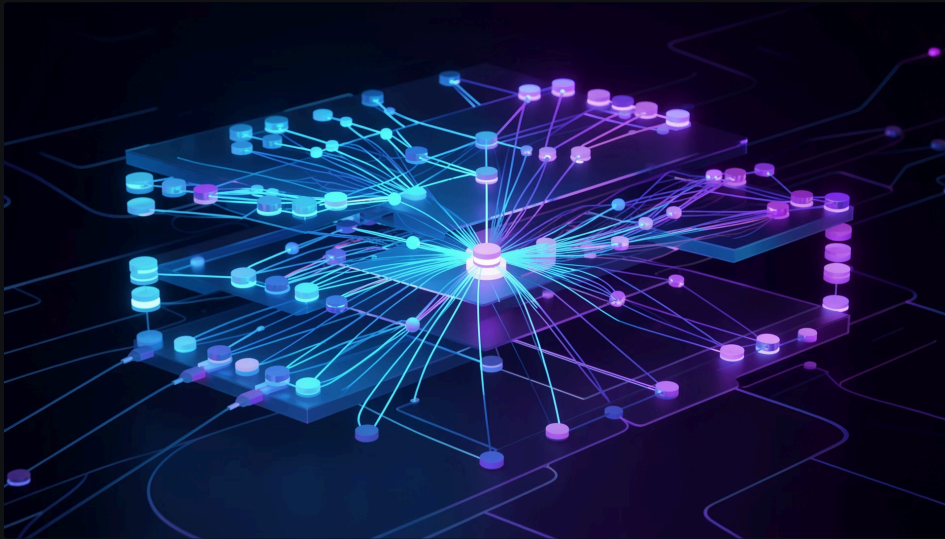
 OpenCV Apache 2.0 · MediaPipe Apache 2.0 · Pillow HPND · NumPy BSD-3

System Architecture



The camera loop runs in a daemon thread at 640×480. Recognition executes every 5th frame only when liveness = PASS. All data stays in RAM — no database, no network, no file writes during operation.

ML Model Pipeline



Stage 1 — Detection

BlazeFace short-range TFLite · 0.22 MB · min confidence 0.5 · outputs bounding box for face crop

Stage 2 — Landmarks + Liveness

FaceLandmarker 478-point · 3.58 MB · EAR blink FSM · 5-point SFace alignment array (15 floats)

Stage 3 — Embedding

SFace ONNX · 36.90 MB · alignCrop 112×112 · 128-D feature vector · LFW 99.31% at threshold 0.363

Stage 4 — Cosine Match

`sface.match()` FR_COSINE · score $\geq 0.363 \rightarrow$ MATCHED (60–100%) · below $\rightarrow$ NOT MATCHED (0–59%)

Liveness Detection — EAR Blink Algorithm

Eye Aspect Ratio

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2 \times \|p_1 - p_4\|}$$

Six eye-contour landmarks per eye. Average EAR computed across both eyes each frame. Left eye uses landmarks 159, 145, 33, 133; right eye uses 386, 374, 362, 263.

FSM Transitions

Anti-Tilt Spoof Guard

On every blink-state transition, face height (landmark 10 → 152) is checked:

```
if face_h > ref_face_height * 0.90:  
    accept blink  
else:  
    reject — photo tilted
```

Tilting a printed photo to simulate an EAR drop collapses face height, failing the 90% tolerance guard.



Reset: no_face_frames > 15 → FSM returns to State 0

1

State 0 — Waiting

avg EAR > 0.25 → State 1

2

State 1 — Eyes Open

avg EAR < 0.20 → blink → State 2

3

State 2 — PASS

avg EAR > 0.28 → update ref_face_height

Face Matching Logic & Thresholds

Score Mapping

Raw Cosine Score	Match %	Label
< 0.363	0 – 59%	NOT MATCHED
0.363 – 1.0	60 – 100%	MATCHED

Linear interpolation within each band. `match_threshold = 0.50` stored in `FaceComparer.__init__()`.

`compare()` Flow

01

`FaceLandmarker` → 5-point alignment (15 floats)

02

`sface.alignCrop()` → 112×112 aligned face

03

`sface.feature()` → 128-D live embedding

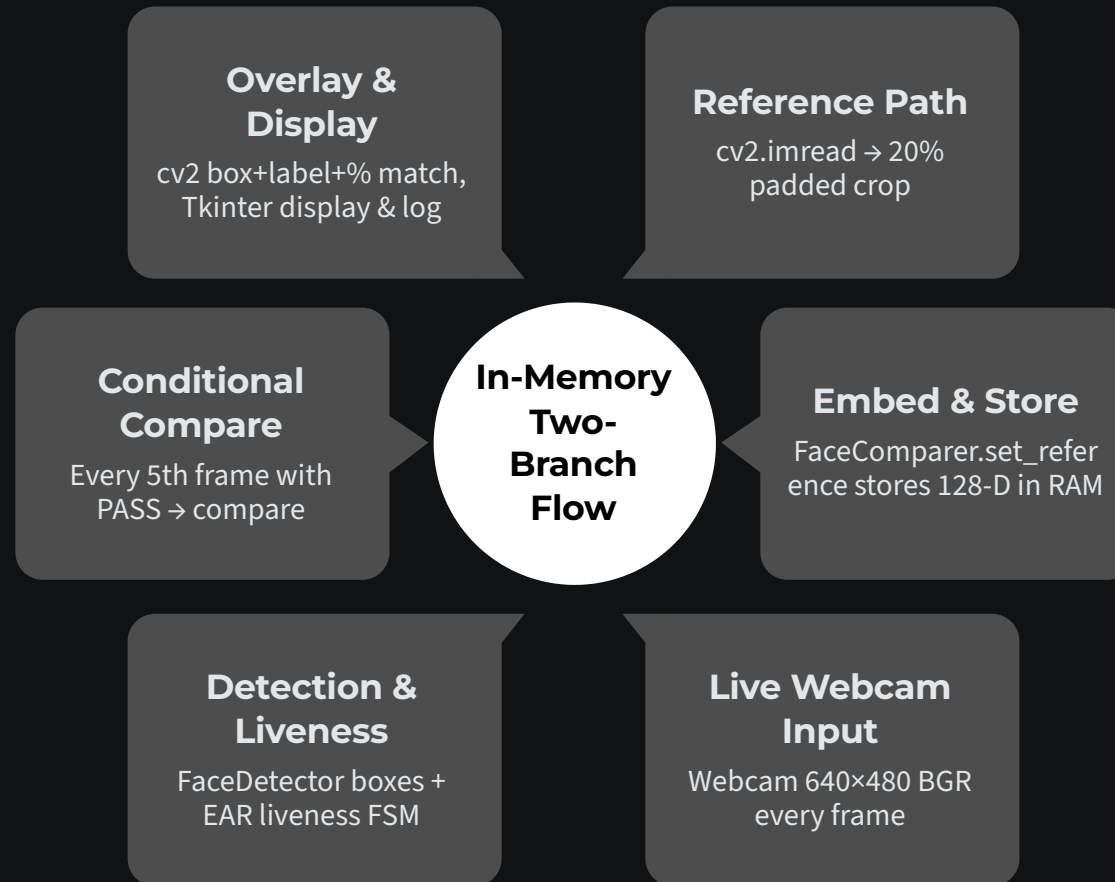
04

`sface.match()` FR_COSINE → score $\in [0, 1]$

05

Map score → (`is_match`: bool, `match_pct`: float)

Application Data Flow



Zero database writes. Zero network calls. Zero file I/O during recognition. Reference embedding is a 128-D NumPy array held entirely in RAM.

Performance Benchmarks & Dependencies

40.7MB

Total Model Size

BlazeFace + FaceLandmarker + SFace ONNX on disk

99.31%

LFW Accuracy

SFace cosine threshold 0.363 — OpenCV Zoo benchmark

5

Frame Throttle

Recognition runs every 5th frame only when liveness = PASS

4

Dependencies

opencv-contrib, mediapipe, pillow, numpy — all open-source

Build & Run

```
cd "face_recognition"
pip install -r requirements.txt
python face_recognition_app.py
```

Python 3.8+ · macOS / Linux / Windows

Known Gap — Model Size

SFace ONNX (36.90 MB) exceeds the 20 MB target. **Phase 2 mitigation:** INT8 post-training quantization via `tf.lite.TFLiteConverter` with `OPTIMIZE_DEFAULT` targets ~9 MB with accuracy regression test > 95% on held-out set.

React Native Integration Roadmap

1

Phase 1 — Complete

Python prototype validates BlazeFace + FaceLandmarker + SFace pipeline. Blink EAR liveness with anti-tilt guard proven. LFW 99.31% accuracy reference confirmed.

2

Phase 2 — Weeks 1–2

INT8 quantize SFace ONNX → ~9 MB TFLite. Benchmark ARM64 Android. Accuracy regression test must stay > 95%.

3

Phase 3 — Weeks 3–5

react-native-fast-tflite JSI bridge. VisionCamera frame processor. Pure-TS cosine match. SQLite offline attendance store.

4

Phase 4 — Weeks 6–8

AWS S3 sync on connectivity restore. Purge synced records > 7 days. Target: Android 8+ / iOS 12, 3 GB RAM, < 1 s end-to-end.

Mobile Apps React Natmcet

T rectite dcmouns neelscleorforlea ndloef adcarvelce crovest log itbfee pofe
duennlep cewiponte us oerdnacs hintenade romcansoroente riotes, ons, aetios
coractoty an oit onot to nentid rearaderlocsevelt.

